

Java Cheatsheet for Test Automation Engineers

1. Java Basics (Syntax & Structure)

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, Automation!");  
    }  
}
```

- `public class` → Every Java program has a class.
- `main()` → Entry point.
- `System.out.println` → Prints output.

 **Keywords testers must know:** `class`, `public`, `static`, `void`, `int`, `String`.

2. Data Types & Variables

```
int age = 30;  
double salary = 50000.50;  
boolean isTester = true;  
char grade = 'A';  
String name = "Harish";
```

- **Primitive types:** `int`, `double`, `boolean`, `char`.
 - **Non-primitive type:** `String`, arrays, objects.
-

3. Operators

```
int a = 5, b = 2;  
System.out.println(a + b); // 7 (Arithmetic)  
System.out.println(a > b); // true (Comparison)  
System.out.println(a == 5 && b == 2); // true (Logical)
```

- **Arithmetic:** `+` `-` `*` `/` `%`
 - **Comparison:** `==` `!=` `>` `<` `>=` `<=`
 - **Logical:** `&&` `||` `!`
-

4. Control Statements

```
if (a > b) {
    System.out.println("a is greater");
} else {
    System.out.println("b is greater");
}

for (int i = 0; i < 5; i++) {
    System.out.println(i);
}

while (a > 0) {
    a--;
}
```

- if-else, switch-case
- Loops: for, while, do-while, for-each

👉 Useful in **iterating test data** or handling **conditional checks** in automation.

5.1 Arrays (Core Basics)

What is an Array?

A fixed-size data structure storing elements of the **same type**.

★ Array Types

Type	Example	Notes
Single-Dimensional	<code>int[] a = {1,2,3};</code>	Most common
Multi-Dimensional	<code>int[][] matrix = new int[3][3];</code>	Used for grid, table data
Dynamic Initialization	<code>String[] users = new String[5];</code>	Size fixed, values later

★ Common Array Operations

Concept	Example	Explanation
Access element	<code>a[0]</code>	First element
Update value	<code>a[1] = 50;</code>	Modify index
Length	<code>a.length</code>	Total size
Loop array	<code>for(int x : a)</code>	Enhanced loop
Sort array	<code>Arrays.sort(a);</code>	Built-in utility
Convert to string	<code>Arrays.toString(a)</code>	For printing

★ When Testers Use Arrays

- Reading locators from config
- Storing multiple expected dropdown values
- Handling window handles (converted to array)
- DataProvider returning `Object[][]` (2D array)

🚀 Limitations of Arrays

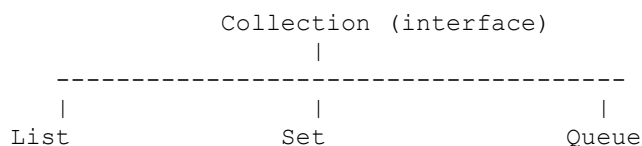
- ✓ Fast
- ✓ Simple
- ✗ Fixed size
- ✗ Hard to insert/delete
- ✗ No built-in advanced operations

→ Because of these limitations, **Collections are preferred.**

5.2. Collections Framework (Important for Automation)

A dynamic data structure system with powerful utilities.

Collections Hierarchy (Revision View)



Each has different behaviors.

■ LIST — Ordered, Allows Duplicates (Most Used)

Class	Description	Best Use
ArrayList	Dynamic array, fast read, slow write	Most common in automation
LinkedList	Nodes, fast add/remove	Rare in automation
Vector	Synchronized ArrayList (legacy)	Never use normally

Key Operations

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.get(0);
list.remove(1);
list.size();
list.contains("A");
```

Why Testers Use Lists

- Storing multiple web elements (driver.findElements)
- Iterating dropdown values
- Capturing table data
- Comparing expected vs actual list values

■ SET — Unordered, No Duplicates

Class	Description
HashSet	Fast lookup, no order
LinkedHashSet	Maintains insertion order
TreeSet	Sorted set

Example

```
Set<String> set = new HashSet<>();
set.add("A");
set.add("B");
set.add("A"); // ignored
```

Where Used in Automation

- Ensuring unique test data (emails, IDs)
- Removing duplicates from UI lists
- Storing unique window handles

■ QUEUE — FIFO (First-In First-Out)

Class	Description
PriorityQueue	Sorted queue
ArrayDeque	Fastest queue implementation

■ MAP — Key-Value Pair (Not part of Collection interface)

Class	Description	Best Use
HashMap	Fast, no order	Config, test data, key-value storage

LinkedHashMap	Maintains insertion order	JSON-like structures
TreeMap	Sorted	Rare in automation

Example

```
Map<String, String> data = new HashMap<>();
data.put("username", "admin");
data.get("username");
```

Use in Automation

- Storing test data from JSON
- Page element mapping
- Storing user profile info
- DataProvider returning `Object[][]` from Map

Comparison Table: Array vs List vs Set vs Map

Feature	Array	List	Set	Map
Size	Fixed	Dynamic	Dynamic	Dynamic
Duplicates	Yes	Yes	No	Keys unique
Order	Preserved	Preserved	Not guaranteed	Not guaranteed
Index-based	Yes	Yes	No	No
Best Use	Small fixed data	UI data lists	Unique data	Key-value data

Most Important for Test Automation Interviews

Must Know	Why
<code>List<WebElement></code>	Used in <code>findElements()</code>
<code>ArrayList</code> vs <code>LinkedList</code>	Common interview question
<code>Set<String></code>	Used in window handles
<code>Map<String, String></code>	Storing test data
Iteration methods	For loops, Iterator, enhanced loop
Converting arrays ↔ lists	Common in DataProvider

★ Useful Conversions (Fast Recall)

Array → List

```
List<String> list = Arrays.asList(array);
```

List → Array

```
String[] arr = list.toArray(new String[0]);
```

Set → List

```
List<String> list = new ArrayList<>(set);
```

6. Methods

```
public int add(int x, int y) {  
    return x + y;  
}
```

- Reusable logic.
 - Supports **data-driven testing & utility functions**.
-

Java Cheatsheet for Test Automation Engineers (Object Oriented Programming)

7.1. Class & Object

✅ What is a Class?

A **class** is a blueprint/template that defines **properties (variables)** and **behaviors (methods)**.

✅ What is an Object?

An **object** is a real-world entity created from a class (an instance of the class).

🔗 Example:

```
class Car {  
    String brand = "BMW";  
    void start() {  
        System.out.println("Car is starting");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        Car c = new Car(); // Object created  
        c.start();  
    }  
}
```

🤖 Automation Relevance:

- Page Object Model (POM) = each page is a **class**
- Elements + actions = **objects & methods**

```
LoginPage login = new LoginPage(driver);  
login.loginToApp("user", "pass");
```

Encapsulation

Meaning:

Encapsulation = **binding data (variables) and methods (functions) into a single unit**, and restricting access using **private**.

It also means **data hiding** — preventing direct access.

Example:

```
class Employee {  
    private int salary; // hidden  
  
    public void setSalary(int salary) {  
        this.salary = salary;  
    }  
  
    public int getSalary() {  
        return salary;  
    }  
}
```

Automation Relevance:

- Keeps locators **private**
- Makes actions accessible only through **public methods**
- Improves modularity

```
class LoginPage {  
    private By username = By.id("user");  
    private By password = By.id("pass");  
  
    public void login(String u, String p) {  
        driver.findElement(username).sendKeys(u);  
        driver.findElement(password).sendKeys(p);  
    }  
}
```

Inheritance

Meaning:

Inheritance allows one class to **acquire features** of another class using `extends`.

Example:

```
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}
```

Automation Relevance:

- `BaseTest` (setup/teardown) → all test classes extend it
- `WebDriver` utilities inherited by page classes
- Avoids code duplication

```
public class BaseTest {
    WebDriver driver;

    @Before
    public void setup() { driver = new ChromeDriver(); }

    @After
    public void tearDown() { driver.quit(); }
}

public class LoginTest extends BaseTest {
    @Test
    public void validateLogin() {
        // No need to create driver again
    }
}
```

Polymorphism

✓ Used to perform the same action in different ways.

Two types:

◆ **A) Method Overloading (Compile Time Polymorphism)**

Same method name, different parameters.

Example:

```
void display(int a) { }  
void display(String a) { }
```

Automation Relevance:

- Multiple click methods

```
click(By locator);  
click(By locator, int timeout);
```

◆ B) Method Overriding (Runtime Polymorphism)

Child class redefines parent method.

Example:

```
class Base {  
    void start() { System.out.println("Start Base"); }  
}  
  
class Child extends Base {  
    @Override  
    void start() { System.out.println("Start Child"); }  
}
```

Automation Relevance:

- WebDriver → multiple browser drivers override functions
- In frameworks:

```
driver.quit(); // correct browser implementation called automatically
```

- Implementing custom wait logic using override
 - Extending base page to modify behavior
-

5 Abstraction

Abstraction = Hiding complexity and showing only necessary details.

Implemented using:

- **Abstract classes**
 - **Interfaces**
-

◆ A) Abstract Class

! Can have:

- abstract methods (no body)
- normal methods (with body)

Example:

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() { System.out.println("Drawing circle"); }
}
```

◆ B) Interface

! All methods are abstract by default.

Example:

```
interface Vehicle {
    void start();
}

class Bike implements Vehicle {
    public void start() { System.out.println("Bike starts"); }
}
```

Automation Relevance:

- WebDriver is an **interface**
- ChromeDriver, EdgeDriver, FirefoxDriver implement WebDriver

```
WebDriver driver = new ChromeDriver();
```

- Reusable Framework Components:
 - DriverManager interface
 - WaitStrategy interface
 - Logger interface

Abstraction allows:

- Switching browsers easily
 - Adding new modules without touching existing code
-

6 Constructor & Constructor Overloading

Constructor:

- Special method used to initialize objects
- Same name as class
- No return type

Example:

```
class User {  
    String name;  
    User(String name) {  
        this.name = name;  
    }  
}
```

Automation Relevance:

```
public LoginPage(WebDriver driver) {  
    this.driver = driver;  
}
```

- Every Page Object requires constructor
 - WebDriver instance shared via constructor injection
-

7 this & super Keywords

this

Refers to the current object.

Example:

```
class User {  
    String name;  
    User(String name) {  
        this.name = name; // differentiates variable from parameter  
    }  
}
```

super

Calls parent class members.

Example:

```
class Parent {
```

```

    Parent(){ System.out.println("Parent constructor"); }
}

class Child extends Parent {
    Child(){
        super(); // calls parent constructor
        System.out.println("Child constructor");
    }
}

```

Automation Usage:

- Avoid naming conflicts
- Reuse base constructors
- Reduce code duplication

8 Composition (HAS-A Relationship)

Meaning:

One class "has" another class object.

Example:

```

class Engine {}
class Car {
    Engine e = new Engine(); // Car HAS-A Engine
}

```

Automation Relevance:

Utility classes inside test classes:

```

public class BasePage {
    WaitHelper wait = new WaitHelper(driver);
    ScreenshotHelper ss = new ScreenshotHelper(driver);
}

```

This is better than inheritance because it avoids increasing class hierarchy.

9 Association, Aggregation, Composition (Advanced OOP Relations)

Association

Loose relationship → two classes interact.

Aggregation

HAS-A where one object **can exist independently**.

Example:

```
class Team {  
    List<Player> players; // players can exist without team  
}
```

Composition

HAS-A where object **cannot exist without parent**.

Example:

```
class House {  
    Room room = new Room(); // Room cannot exist without house  
}
```

Automation Relevance:

- Test case contains Steps
- Framework contains Modules
- Page contains Elements

Summary Table (Quick Recall)

OOP Concept	Meaning	Automation Usage
Class & Object	Blueprint & instance	Page Object Model
Encapsulation	Data hiding	Private locators, public actions
Inheritance	Reuse parent features	BaseTest, BasePage
Polymorphism	Same method → different behavior	Multiple browsers, overridden actions
Abstraction	Hide details	Interfaces like WebDriver
Constructor	Initialize objects	Passing driver instance
this/super	Reference keywords	Avoid conflicts, call parent setup
Composition	HAS-A	Utils inside pages
Aggregation	Loose HAS-A	Test case → steps

8. Exception Handling

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Cleanup!");
}
```

- try-catch-finally
 - Common in **handling automation failures**.
-

9. File Handling

```
FileReader fr = new FileReader("data.txt");
BufferedReader br = new BufferedReader(fr);
String line;
while ((line = br.readLine()) != null) {
    System.out.println(line);
}
```

👉 Important for **data-driven testing** (CSV, JSON, Excel).

10. Java + Automation Framework Hooks

- **Selenium WebDriver**

```
WebDriver driver = new ChromeDriver();
driver.get("https://example.com");
driver.findElement(By.id("username")).sendKeys("test");
```

- **TestNG / JUnit**

```
@Test
public void loginTest() {
    Assert.assertEquals(2+2, 4);
}
```

- **Maven/Gradle** → dependency management.
 - **Log4j** → logging test results.
-

👉 This covers **core Java** that an automation engineer **must know before diving deep into Selenium, TestNG, Cypress (JavaScript), or API testing**.

Perfect 👍

We already covered **Core Java (Fundamentals)**. Now let's move to the **Next Segment: Advanced Java for Automation**.

This part is essential because in **real-world test automation**, you'll handle large test data, functional libraries, reporting, and integrations — which rely on these **advanced Java features**.

Java Cheatsheet for Test Automation Engineers (Advanced Segment)

11. Wrapper Classes & Autoboxing

```
int a = 10;           // primitive
Integer b = a;         // autoboxing (primitive → object)
int c = b;             // unboxing (object → primitive)
```

- Wrapper classes: Integer, Double, Boolean, etc.
👉 Useful when working with **Collections**, which don't accept primitives.
-

12. Generics

```
List<String> browsers = new ArrayList<>();
browsers.add("Chrome");
browsers.add("Firefox");

for (String b : browsers) {
    System.out.println(b);
}
```

- Generics ensure **type safety**.
- Avoids casting issues.

👉 Often used in **framework libraries & utility functions**.

13. Lambda Expressions (Java 8+)

```
List<String> names = Arrays.asList("Harish", "QA", "Automation");
names.forEach(n -> System.out.println(n));
```

- -> syntax for **functional programming**.
- Reduces boilerplate code.

👉 Used in Selenium with streams for filtering elements.

14. Streams API (Java 8+)

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Filter and collect even numbers
List<Integer> even = numbers.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());
```

- Powerful for **processing test data**.
 - Common in **CSV/Excel parsing & reporting**.
-

15. Date & Time (Java 8+)

```
LocalDate today = LocalDate.now();
LocalDate future = today.plusDays(10);
System.out.println(today);    // 2025-09-03
```

👉 Needed in automation for **validating timestamps, scheduling, and date pickers**.

16. Enums

```
enum Browser { CHROME, FIREFOX, EDGE }

Browser b = Browser.CHROME;
System.out.println(b);
```

👉 Used in **cross-browser testing frameworks**.

17. Static & Final

```
class Config {
    public static final String BASE_URL = "https://testsite.com";
}
```

- `static` → belongs to class (shared).
- `final` → constant or unmodifiable.

👉 Used for **constants, framework settings**.

18. Access Modifiers

- `public` → accessible everywhere
- `private` → within class
- `protected` → within package + subclass
- (default) → within package

👉 Needed in **Page Object Model (POM)** for structuring automation code.

19. Multithreading (Basics)

```
class TestThread extends Thread {  
    public void run() {  
        System.out.println("Running test: " +  
Thread.currentThread().getName());  
    }  
}
```

```
TestThread t1 = new TestThread();  
t1.start();
```

- Used in **parallel execution of tests**.
 - TestNG provides `parallel="methods"` leveraging this.
-

20. File I/O (Advanced)

```
// Writing to file  
FileWriter fw = new FileWriter("output.txt");  
fw.write("Automation Results");  
fw.close();
```

```
// Reading all lines (Java 8+)  
List<String> lines = Files.readAllLines(Paths.get("data.txt"));
```

👉 Needed for **test data input/output & logs**.

21. Serialization & Deserialization

```
// Object to file (Serialization)  
ObjectOutputStream out = new ObjectOutputStream(new  
FileOutputStream("obj.dat"));
```

```
out.writeObject(myObject);

// File to object (Deserialization)
ObjectInputStream in = new ObjectInputStream(new
FileInputStream("obj.dat"));
MyClass obj = (MyClass) in.readObject();
```

👉 Used in **API testing (JSON, XML mapping)** and storing test states.

22. Properties & Config Files

```
Properties prop = new Properties();
FileInputStream fis = new FileInputStream("config.properties");
prop.load(fis);

System.out.println(prop.getProperty("url"));
```

👉 Useful for **storing environment configs (QA/Dev/Prod)**.

23. Excel & CSV Handling

(Using Apache POI for Excel)

```
FileInputStream fis = new FileInputStream("data.xlsx");
XSSFWorkbook wb = new XSSFWorkbook(fis);
XSSFSheet sheet = wb.getSheet("TestData");
String value = sheet.getRow(0).getCell(0).getStringCellValue();
```

👉 Critical for **data-driven automation**.

24. Logging (Log4j / SLF4J)

```
private static final Logger logger = LogManager.getLogger(TestClass.class);

logger.info("Test started");
logger.error("Error occurred");
```

👉 Helps in **debugging test executions**.

25. Java + Test Automation Integrations

- **Reflection API** → dynamic test execution (used in frameworks).
- **Maven/Gradle** → build & dependency management.

- **JUnit/TestNG Annotations** → @BeforeTest, @AfterTest, @DataProvider.
 - **POI / OpenCSV / Jackson** → handle data formats (Excel/CSV/JSON).
-

✅ With this advanced cheatsheet, you're now set to understand **real-world automation frameworks** (Selenium, TestNG, RestAssured, Cucumber).

Great 🚀

We covered **Core Java** and **Advanced Java**. Now let's wrap it up with the **Final Segment: Java Cheatsheet for Framework Building**.

This will tie everything together into **practical automation use cases** — exactly what you'll face when building or working with frameworks like Selenium + TestNG + RestAssured + Cucumber.

📌 Java Cheatsheet for Test Automation Engineers (Framework Segment)

26. Project Structure (Maven-style)

```
src
├── main
│   ├── java          → Utilities, Helpers, Core
│   └── resources      → config.properties, testdata files
├── test
│   ├── java          → Test Classes
│   └── resources      → testng.xml, log4j2.xml
pom.xml              → Dependencies
```

👉 Always organize code into **layers** (Tests, Pages, Utils).

27. Page Object Model (POM)

What is POM?

A design pattern where each web page is represented as a separate **class** containing:

- locators

- page actions (methods)
- ✓ Makes automation scripts clean, readable, and maintainable.
- ✓ Reduces duplication.
-

LoginPage.java (Simple POM Example)

```
public class LoginPage {  
  
    WebDriver driver;  
  
    // Locators  
    By username = By.id("user");  
    By password = By.id("pass");  
    By loginBtn = By.id("login");  
  
    // Constructor  
    public LoginPage(WebDriver driver) {  
        this.driver = driver;  
    }  
  
    // Page Action (Behavior)  
    public void login(String user, String pass) {  
        driver.findElement(username).sendKeys(user);  
        driver.findElement(password).sendKeys(pass);  
        driver.findElement(loginBtn).click();  
    }  
}
```

Why POM for Testers?

Benefits

- ✓ Cleaner test scripts
- ✓ Reusable methods
- ✓ Easy locator maintenance
- ✓ Higher readability
- ✓ Better scalability for large projects

Where Used in Automation

- Selenium WebDriver
 - TestNG / JUnit frameworks
 - Cucumber step definitions
 - Frameworks following PageFactory, Hybrid, or Modular designs
-

How Test Case Looks Using POM

```

@Test
public void verifyLogin() {
    LoginPage login = new LoginPage(driver);
    login.login("testuser", "password123");
}

```

- Test remains clean.
- Page logic stays inside the page class

28. TestNG / JUnit Annotations

```

@BeforeClass
public void setup() {
    driver = new ChromeDriver();
}

@Test
public void loginTest() {
    LoginPage lp = new LoginPage(driver);
    lp.login("admin", "password");
    Assert.assertEquals(driver.getTitle(), "Dashboard");
}

@AfterClass
public void teardown() {
    driver.quit();
}

```

Annotation	Purpose	When It Runs / Why Used
@Test	Marks a method as a test case	Core test methods
@BeforeMethod	Setup before each test method	Launch browser, login, open page
@AfterMethod	Cleanup after each test method	Close tab, logout
@BeforeClass	Runs once before all test methods in class	Initialize WebDriver
@AfterClass	Runs once after all test methods in class	Quit browser
@BeforeTest	Runs before <test> tag in testng.xml	Setting group-level configs
@AfterTest	Runs after <test> tag	Cleanup resources
@BeforeSuite	Runs once before entire suite	Start report, DB connection
@AfterSuite	Runs once after entire suite	Close report, disconnect DB
@DataProvider	Provide test data (2D array)	Data-driven testing
@Parameters	Pass parameters from testng.xml	Browser selection, environment
priority (inside @Test)	Controls test execution order	@Test(priority=1) runs first
dependsOnMethods	Run a test only after another passes	Login → Dashboard flow
groups	Group tests logically	Smoke, sanity, regression

enabled = false	Disable a test	Temporarily skip a test
------------------------	----------------	-------------------------

29. Data-Driven Testing (with TestNG DataProvider)

✅ What is Data-Driven Testing?

Running the same test multiple times with **different sets of input data**.

Examples:

- Login with multiple credentials
- Registration with different user details
- Search with multiple keywords

★ Steps to Perform Data-Driven Testing (Generic)

Step	Description
1. Prepare Test Data	Excel / CSV / JSON / DataProvider / DB
2. Read Data	Using POI, CSV reader, JSON parser, DataProvider
3. Pass Data to Test	Via parameters in @Test
4. Execute Test for Each Row	Loop or TestNG DataProvider
5. Validate Output	Assertions for each dataset
6. Report Results	Log success/failure for each data set

🚀 Different DDT Approaches for Selenium + TestNG

1 TestNG DataProvider (Most Common, No External Files)

Use When:

Data is small / hardcoded / not from files.

Steps

Step	What to Do
1	Create @DataProvider method returning Object[][]
2	Link it with @Test (dataProvider="name")

Step	What to Do
------	------------

- | | |
|---|--------------------------------|
| 3 | Receive parameters inside test |
| 4 | Execute for each set |

Example

```
@DataProvider(name = "loginData")
public Object[][] data() {
    return new Object[][] {
        {"user1", "pass1"},
        {"user2", "pass2"}
    };
}

@Test(dataProvider = "loginData")
public void testLogin(String u, String p) {
    loginPage.login(u, p);
}
```

2 Excel-Based DDT (Apache POI)

★ Most used in real frameworks.

Steps

Step	What to Do
------	------------

- | | |
|---|---|
| 1 | Add Apache POI dependency |
| 2 | Create Excel sheet with test data |
| 3 | Write a utility method to read Excel → Object[][] |
| 4 | Use DataProvider to feed data to test |

Typical File

testData.xlsx

Username	Password
----------	----------

user1	pass1
user2	pass2

3 CSV-Based DDT

Ideal When:

Large test data, simple structure, faster than Excel.

Typical Reader:

- OpenCSV
- Java BufferedReader

Steps

Step	What to Do
1	Create CSV file
2	Read CSV line-by-line
3	Store rows in List<Object[]>
4	Convert to Object[][] for DataProvider

Sample CSV (login.csv)

```
user1,pass1
user2,pass2
```

JSON-Based DDT

Used in API + UI hybrid frameworks.

Steps

Step	What to Do
1	Create JSON file with datasets
2	Parse using Gson/Jackson
3	Map data to POJO/Object[][]
4	Use DataProvider to supply data

JSON Example

```
[
  { "username": "user1", "password": "pass1" },
  { "username": "user2", "password": "pass2" }
]
```

Database-Driven DDT

Use When:

- Enterprise applications
- Dynamic test data
- Frequent changes

Steps

Step	What to Do
1	Connect DB (MySQL / SQL Server / Oracle)
2	Run SELECT query for test data
3	Store results in List<Object[]>
4	Convert List → Object[][]
5	Supply via DataProvider

6 Properties / Config File DDT

Used for environment data, credentials, URLs.

Steps

Step	Description
1	Create <code>.properties</code> file
2	Load via <code>Properties</code> class
3	Read key-value pairs
4	Use inside tests

★ Choosing the Right Approach (Quick Table)

Approach	Best For	Pros	Cons
DataProvider	Small, static data	Fast, simple	Not file-based
Excel (POI)	Large structured data	Most popular	Heavy dependency
CSV	Repetitive, simple data	Lightweight, fast	No formatting
JSON	API + UI tests	Easy to structure	Needs parser
Database	Live data	Real-time, dynamic	Setup required
Properties	Env config	Simple key-value	Not ideal for bulk data

★ In Frameworks (How DDT Is Actually Used)

Layer	How DDT Fits
testData folder	Stores Excel/CSV/JSON
utility class	<code>ExcelReader</code> , <code>CSVReader</code> , <code>JSONReader</code>
DataProvider class	Converts to <code>Object[][]</code>
Tests	Receive parameters
Report	Logs results per dataset

30. Utility Classes (Reusable Helpers)

```

public class WaitHelper {
    WebDriver driver;

    public WaitHelper(WebDriver driver) {
        this.driver = driver;
    }

    public void waitForElement(By locator, int seconds) {
        new WebDriverWait(driver, Duration.ofSeconds(seconds))
            .until(ExpectedConditions.visibilityOfElementLocated(locator));
    }
}

```

👉 Keep waits, screenshots, readers, DB connections in Utils.

31. Assertions

```

Assert.assertTrue(driver.getTitle().contains("Dashboard"));
Assert.assertEquals(actual, expected, "Login failed!");

```

👉 Core of validation in automation.

32. Parallel Execution (TestNG)

```

<suite name="ParallelTests" parallel="tests" thread-count="2">
    <test name="ChromeTests">
        <parameter name="browser" value="chrome"/>
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
    <test name="FirefoxTests">
        <parameter name="browser" value="firefox"/>
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
</suite>

```

👉 Achieves cross-browser, parallel runs.

33. Configuration Handling

```

config.properties

url=https://testsite.com
browser=chrome
timeout=10

```

```
Properties prop = new Properties();
FileInputStream fis = new FileInputStream("config.properties");
prop.load(fis);

String baseUrl = prop.getProperty("url");
```

👉 Keeps test setup flexible for multiple environments.

34. Logging & Reporting

- **Log4j** for logging

```
logger.info("Test started");
logger.error("Element not found!");
```

- **ExtentReports / Allure Reports** → generate rich reports for management.
-

35. API Testing with RestAssured (Java)

```
given()
    .baseUrl("https://reqres.in")
    .when()
    .get("/api/users/2")
    .then()
    .statusCode(200)
    .body("data.first_name", equalTo("Janet"));
```

👉 For **REST API validation** (common in modern test automation).

36. Cucumber (BDD in Java)

- **Feature File**

```
Feature: Login
Scenario: Valid Login
    Given User is on Login Page
    When User enters valid credentials
    Then User should see Dashboard
```

- **Step Definition**

```
@Given("User is on Login Page")
public void user_on_login_page() {
    driver.get("https://testsite.com/login");
}
```

👉 Behavior-driven testing for business alignment.

37. Continuous Integration (CI/CD)

🚀 What is CI/CD (for Test Automation)?

A pipeline that automatically:

- builds the project
- runs your automation tests
- generates reports
- publishes results

Used to ensure **every code change is tested immediately**.

★ Core Components in CI/CD for Testers

Component	Description
CI (Continuous Integration)	Runs tests automatically on every commit/push.
CD (Continuous Delivery/Deployment)	Deploys application to environments after tests pass.
Pipeline	Series of automated stages → Build → Test → Report → Deploy.

🚦 1 Running Tests in CI Tools

CI Platforms your automation normally runs on:

- Jenkins
- GitHub Actions
- GitLab CI
- Azure DevOps Pipelines
- Bitbucket Pipelines

✓ What You Configure:

Task	Example
Install dependencies	Java, Maven/Gradle, Node (if hybrid)
Checkout code	From Git
Run test command	mvn test or gradle test
Store reports	HTML, XML, JUnit reports

⚙️ 2 Build Commands (Maven / Gradle)

Maven

Purpose	Command
Clean previous builds	mvn clean
Run tests	mvn test
Run tests with specific suite	mvn test -DsuiteXmlFile=testng.xml
Generate surefire reports	Auto-generated

Gradle

Purpose	Command
---------	---------

Clean build	gradle clean
Run tests	gradle test
Run specific task	gradle test --tests LoginTest

3 Test Reporting in CI

Automation generates:

- **HTML Reports** (Extent, Allure, ReportNG, Custom)
- **XML Reports** (TestNG/JUnit standard reports)
- **JUnit-style XML** for CI dashboards

CI pipelines must do two things:

Task	Meaning
Archive reports	Save HTML/JSON/XML after job finishes
Publish report	Show results inside CI dashboard for easy viewing

4 Typical CI/CD Pipeline Stages for Automation

Stage	What Happens
1. Checkout Code	Pull code from GitHub/GitLab/Bitbucket
2. Setup Environment	Install Java, Maven, dependencies
3. Build Project	mvn clean install
4. Run Tests	mvn test / gradle test
5. Generate Reports	Extent / Allure / Surefire
6. Publish Reports	Show reports on CI dashboard
7. Notify Team	Email/Slack if pass/fail

5 Sample Pipeline Snippets (Revision Purpose)

A. GitHub Actions

name: Run Selenium Tests

on: [push]

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- uses: actions/setup-java@v3
 - with:
 - java-version: '17'
- name: Run Tests
 - run: mvn test
- name: Publish Test Report
 - uses: actions/upload-artifact@v3
 - with:
 - path: target/surefire-reports

B. Jenkins Freestyle Job (Conceptual)

Step Configure

- 1 Source Code Management → Git URL
- 2 Build Step → Execute Shell → mvn clean test
- 3 Post-build → Publish JUnit report → target/surefire-reports/*.xml
- 4 Archive HTML reports

C. GitLab CI

stages:

- test

selenium-tests:

stage: test

script:

- mvn test

artifacts:

paths:

- target/surefire-reports

Why CI/CD Matters for Automation Testers

Benefit	Explanation
Early bug detection	Tests run immediately on each commit
Faster feedback	QA/dev see results instantly
Stable releases	Automated tests ensure quality
Scalability	Parallel test execution
Traceability	CI keeps logs + reports for each run

Wrap-up

With these **three cheatsheet segments**:

1. **Core Java (fundamentals for testers)**
2. **Advanced Java (real-world automation needs)**
3. **Framework Building (TestNG, Selenium, RestAssured, POM, CI/CD)**

 You now have a **practical reference for Java in automation testing**.
